C++ Inheritance Practice Problems

Lecture 4 Supplemental Notes & Solutions

Problem 1 — Basic Inheritance

Code Example

```C#
#include <iostream>
using namespace std;

class Animal {
public:
    void eat() {
        cout << "Animal eats" << endl;
    }
};

class Dog : public Animal {
public:
    void bark() {
        cout << "Dog barks" << endl;
    }
};

int main() {
    Dog d;
    d.eat();
    d.bark();
    return 0;
}
```

Question

What will be the output of the program above?

Answer

```
Animal eats
Dog barks
```

Explanation

Dog is the derived class, and `Animal` is the base class. Due to public inheritance, the `eat()` member function is accessible directly from a Dog object.

Problem 2 — Private Members in Inheritance

Code Example

```C#
#include <iostream>
using namespace std;

class Base {
private:
    int x;
public:
    void setX(int n) { x = n; }
    int getX() { return x; }
};

class Derived : public Base {
public:
    void show() {
        // cout << x;    // ERROR
        cout << getX();  // OK
```

```
    }
};

int main() {
    Derived d;
    d.setX(10);
    d.show();
    return 0;
}
```

Question

Why does attempting to access `cout << x;` directly inside `Derived::show()` cause a compilation error?

Answer

The variable `x` is declared as a **private** member of `Base`. Derived classes cannot directly access private members of their base class. However, because `getX()` is a public member function, it can be used to retrieve the value.

Problem 3 — Protected Access Specifier

Code Example

```
C#
#include <iostream>
using namespace std;

class Base {
protected:
    int x;
public:
```

```
    void setX(int n) { x = n; }
};

class Derived : public Base {
public:
    void show() {
        cout << x;  // Accessible
    }
};

int main() {
    Derived d;
    d.setX(20);
    d.show();
    return 0;
}
```

Question

What will be the output of this program?

Answer

20

Explanation

Because `x` is declared as **protected** in `Base`, members of the `Derived` class can access it directly. However, external code outside the class hierarchy (such as `main()`) still cannot access `d.x` directly.

Problem 4 — Public vs. Private Inheritance

Code Example

```
C#
#include <iostream>
using namespace std;

class Base {
public:
    void show() { cout << "Hello"; }
};

class Derived : public Base {};

int main() {
    Derived d;
    d.show();
    return 0;
}
```

Question

1. Will the public inheritance snippet above compile and execute successfully?

2. What happens if the inheritance access specifier is changed to `private` (i.e. `class Derived : private Base`)?

Answer

1. Public Inheritance: Yes, it compiles and runs. Public members of `Base` remain public in `Derived`.

2. Private Inheritance: Compilation Error. Calling `d.show()` from `main()` will fail.

Explanation

Under **private inheritance**, public and protected members of the base class become private members of the derived class, preventing them from being called on derived class instances from outside the class.

Problem 5 — Constructor and Destructor Execution Order

Code Example

```C#
#include <iostream>
using namespace std;

class Base {
public:
    Base() { cout << "Base Constructor" << endl; }
    ~Base() { cout << "Base Destructor" << endl; }
};

class Derived : public Base {
public:
    Derived() { cout << "Derived Constructor" << endl; }
    ~Derived() { cout << "Derived Destructor" << endl; }
};

int main() {
    Derived d;
    return 0;
}
```

Question

In what order are the constructors and destructors executed when creating and destroying an instance of `Derived`?

Answer

```
Base Constructor
Derived Constructor
Derived Destructor
Base Destructor
```

Explanation

Execution Rules:

• **Constructors:** Executed top-down (Base first, then Derived).

• **Destructors:** Executed bottom-up in reverse order (Derived first, then Base).

Problem 6 — Passing Arguments to Base Class Constructors

Code Example

```
C#
#include <iostream>
using namespace std;

class Base {
public:
    Base(int x) {
        cout << "Base: " << x << endl;
    }
};

class Derived : public Base {
```

```
public:
    Derived(int x) : Base(x) {
        cout << "Derived: " << x << endl;
    }
};

int main() {
    Derived d(10);
    return 0;
}
```

Question

What will be printed when initializing `Derived d(10);`?

Answer

```
Base: 10
Derived: 10
```

Explanation

Using initializer list syntax (`Derived(int x) : Base(x)`), the derived class constructor forwards argument `x` to the parameterized base class constructor before executing its own constructor body.

Problem 7 — Multiple Inheritance Constructor Order

Code Example

```
C#
#include <iostream>
```

```
using namespace std;

class A {
public:
    A() { cout << "A" << endl; }
};

class B {
public:
    B() { cout << "B" << endl; }
};

class C : public A, public B {
public:
    C() { cout << "C" << endl; }
};

int main() {
    C obj;
    return 0;
}
```

Question

What will be the output when instantiating an object of class C?

Answer

A

B

C

Explanation

In multiple inheritance, base class constructors are executed from **left to right** in the order they appear in the class declaration (`class C : public A, public B`).